The first step in meeting this requirement is creating a pipeline with a processor. We need to add a field called `category` with the value `"confidential"` using a `set` processor. Elasticsearch exposes the `_ingest` API to create and test pipelines. We can create a new pipeline using the _ingest/pipeline/<*pipeline_name*> URL. The following code creates a new pipeline with one processor called `set`.

Listing B.1 Creating a confidential-stamping pipeline

```
PUT _ingest/pipeline/confidential_files_pipeline
{
  "description": "Stamp confidential on the file (document)",
  "processors": [
    {
      "set": {
        "field": "category",
        "value": "confidential"
      }
    }
  ]
}
```

We provide multiple processors as a chain.

A set processor sets a new field.

We create an ingest pipeline called `confidential_files_pipeline` pipeline with a single `set` processor. The `set` processor's job is to create a new `category` field with the value `"confidential"`. When a new document is passed through this pipeline, the `set` processor will add a field called `category` to the document on the fly.

Once the pipeline definition is created, executing it stores the pipeline definitions in a cluster state. It is now ready to be put to use. However, we can use a `_simulate` API call to do a dry run before we start indexing the data.

Listing B.2 Creating a top-secret-stamper pipeline

```
POST _ingest/pipeline/confidential_files_pipeline/_simulate
{
  "docs": [{
      "_source": {
        "op_name": "Operation Cobra"
      }
    }
  ]
}
```

Pipeline URL with the _simulate API

The docs array expects the set of documents.

The _source field consists of the additional field.

This code simulates the pipeline execution. We execute a document with one field-value pair on the `confidential_files_pipeline` pipeline. Executing the code does not index the document; instead, it tests the pipeline's logic. This is the response when we simulate the pipeline:

```
{
  "docs": [
    {
      "doc": {
```